

ET 1:02 : mini exam ends (Pitt)

ET 1:15 : lecture resumes (all campuses)

18-661 Introduction to Machine Learning

Multi-Class Regression

Spring 2026

ECE – Carnegie Mellon University

Announcements

- Homework 1 grades have been released on Gradescope. If you notice a grading error, please submit a regrade request through Gradescope by Sunday, February 16. Review the posted solutions carefully before submitting a request.
- Homework 2 is due on Friday, February 20. After this lecture, you should be able to attempt most of the problems (except those on SVMs).

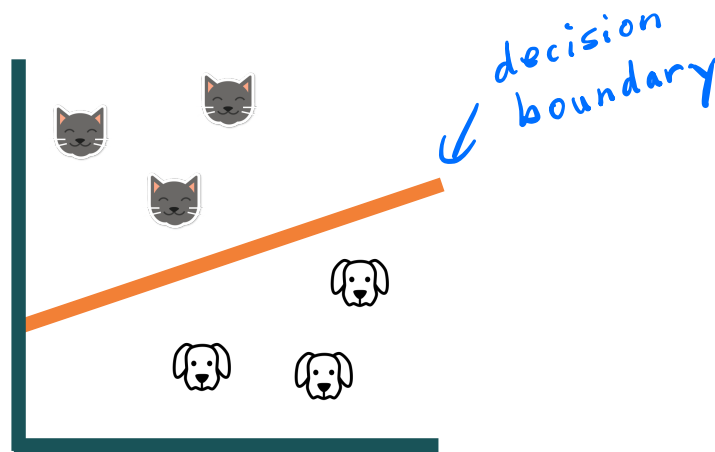
1. Review of Logistic Regression
2. Multi-class Classification

Review of Logistic Regression

Intuition: Logistic Regression

Learn the equation of the decision boundary $\mathbf{w}^\top \mathbf{x} = 0$ such that

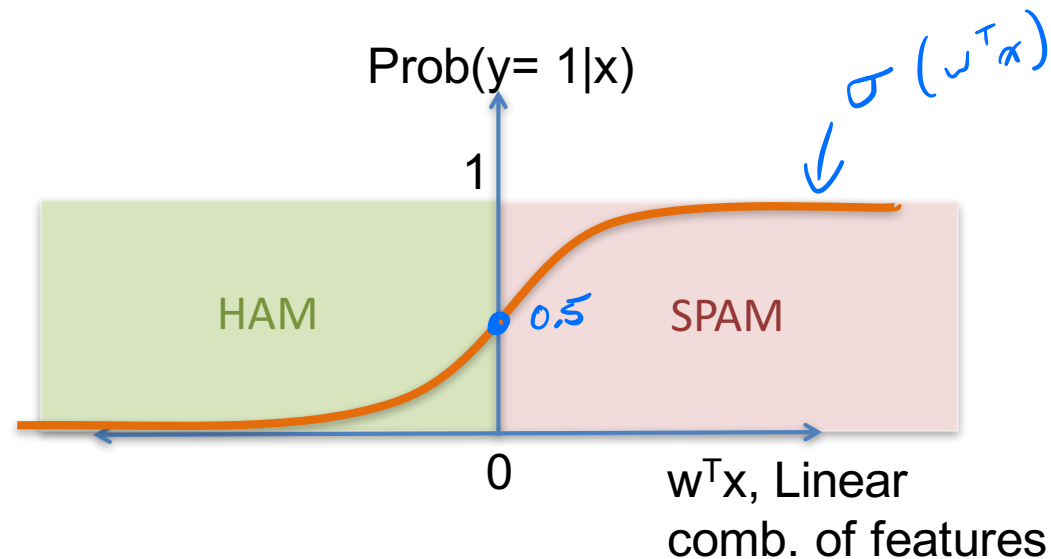
- If $\mathbf{w}^\top \mathbf{x} \geq 0$ declare $y = 1$ (cat)
- If $\mathbf{w}^\top \mathbf{x} < 0$ declare $y = 0$ (dog)



$y = 0$ for dog, $y = 1$ for cat

Intuition: Logistic Regression

- Suppose we want to output the **probability** of an email being spam/ham instead of just 0 or 1
- This gives information about the confidence in the decision
- Use a function $\sigma(\mathbf{w}^\top \mathbf{x})$ that maps $\mathbf{w}^\top \mathbf{x}$ to a value between 0 and 1



Probability that predicted label is 1 (spam)

Key Problem: Finding optimal weights $\mathbf{w}$ that accurately predict this probability for a new email

Formal Setup: Binary Logistic Classification

- Input/features: $\mathbf{x} = [\overset{\text{bias}}{\underset{\circ}{1}}, x_1, x_2, \dots, x_D] \in \mathbb{R}^{D+1}$
- Output: $y \in \{0, 1\}$ *label / target*
- Training data: $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, 2, \dots, N\}$

Formal Setup: Binary Logistic Classification

- Input/features: $\mathbf{x} = [1, x_1, x_2, \dots, x_D] \in \mathbb{R}^{D+1}$
- Output: $y \in \{0, 1\}$
- Training data: $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, 2, \dots, N\}$
- Model:

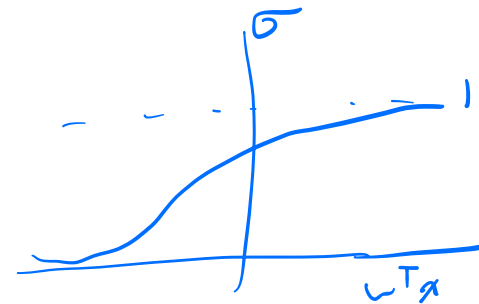
$$p(y = 1 | \mathbf{x}; \mathbf{w}) = \sigma[g(\mathbf{x})]$$

where

$$g(\mathbf{x}) = w_0 + \sum_d w_d x_d = \mathbf{w}^\top \mathbf{x}$$

and $\sigma[\cdot]$ stands for the **sigmoid** function

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



Likelihood Function

Probability of a single training sample $(\mathbf{x}_n, y_n)$...

$$p(y_n|\mathbf{x}_n; \mathbf{w}) = \begin{cases} \sigma(\mathbf{w}^\top \mathbf{x}_n) & \text{if } y_n = 1 \\ 1 - \sigma(\mathbf{w}^\top \mathbf{x}_n) & \text{otherwise} \end{cases}$$

Likelihood Function

Probability of a single training sample $(\mathbf{x}_n, y_n)$...

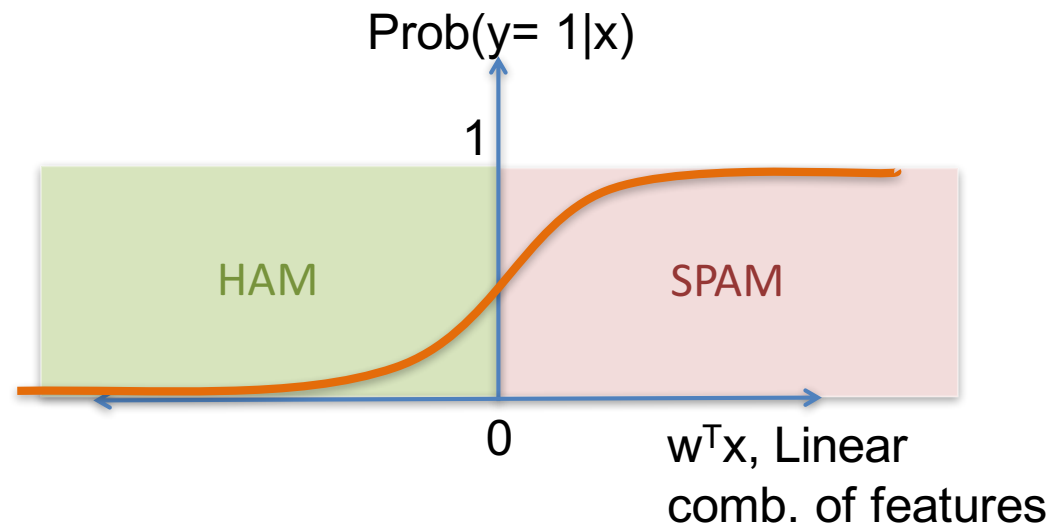
$$p(y_n|\mathbf{x}_n; \mathbf{w}) = \begin{cases} \sigma(\mathbf{w}^\top \mathbf{x}_n) & \text{if } y_n = 1 \\ 1 - \sigma(\mathbf{w}^\top \mathbf{x}_n) & \text{otherwise} \end{cases}$$

equivalent

Simplify, using the fact that y_n is either 1 or 0

$$\underline{p(y_n|\mathbf{x}_n; \mathbf{w})} = \sigma(\mathbf{w}^\top \mathbf{x}_n)^{y_n} [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]^{1-y_n}$$

$y_n \in \{0, 1\}$



Probability that predicted label is 1 (spam)

Log Likelihood or Cross Entropy Error

Log-likelihood of the whole training data $\mathcal{D}$

$$P(\mathcal{D}) = \prod_{n=1}^N p(y_n | \mathbf{x}_n; \mathbf{w}) = \prod_{n=1}^N \{ \sigma(\mathbf{w}^\top \mathbf{x}_n)^{y_n} [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]^{1-y_n} \}$$

maximize

$$\log P(\mathcal{D}) = \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \}$$

*↑
training data*

It is convenient to work with its negation, which is called the

cross-entropy error function

find $\mathbf{w}$ to minimize

negative log likelihood

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \}$$

Cross-Entropy as a Loss Function

We aim to build a function $h(\mathbf{x})$ to predict the true value y associated with $\mathbf{x}$. If we make a mistake, we incur a **loss**

$$\ell(\underbrace{h(\mathbf{x})}_{\text{prediction}}, \underbrace{y}_{\text{true label}})$$

Cross-Entropy as a Loss Function

We aim to build a function $h(\mathbf{x})$ to predict the true value y associated with $\mathbf{x}$. If we make a mistake, we incur a **loss**

$$\ell(h(\mathbf{x}), y)$$

Cross-entropy is also a sum over all data samples:

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \}$$

where $h(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$.

$\ell(\sigma(\mathbf{w}^\top \mathbf{x}), y)$
↑
prediction
of y

Cross-Entropy as a Loss Function

We aim to build a function $h(\mathbf{x})$ to predict the true value y associated with $\mathbf{x}$. If we make a mistake, we incur a **loss**

$$\ell(h(\mathbf{x}), y)$$

Cross-entropy is also a sum over all data samples:

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\}$$

where $h(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$.

What is the loss function?

$$\ell(h(\mathbf{x}_n), y) = -\{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\}$$

Gradient Descent Update for Logistic Regression

Finding the gradient of $\mathcal{E}(\mathbf{w})$ looks very hard, but it turns out to be simple and intuitive.

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\}$$

Gradient Descent Update for Logistic Regression

Finding the gradient of $\mathcal{E}(\mathbf{w})$ looks very hard, but it turns out to be simple and intuitive.

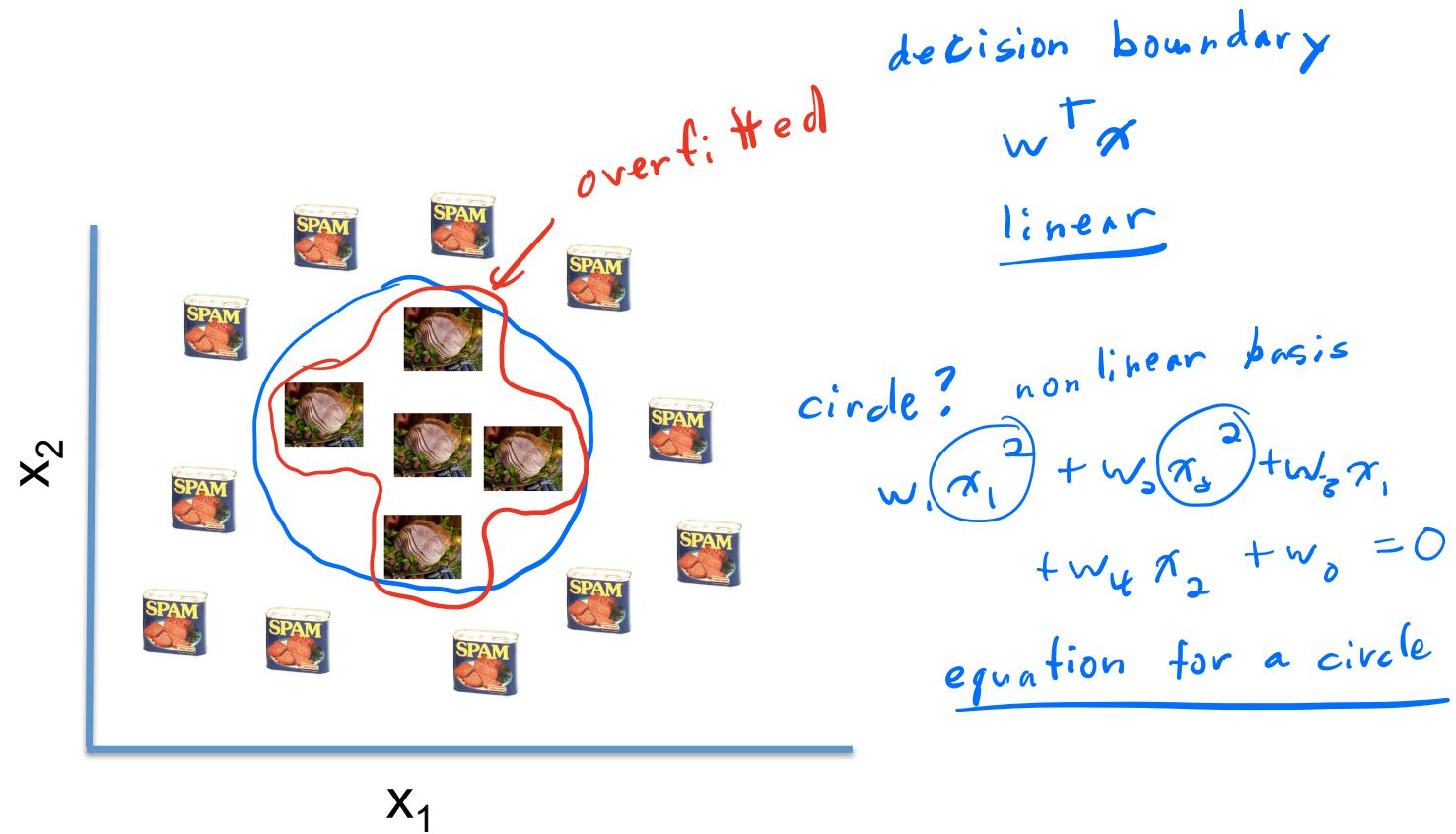
$$\mathcal{E}(\mathbf{w}) = - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\}$$

Computing the gradient

$$\begin{aligned} \frac{\partial \mathcal{E}(\mathbf{w})}{\partial \mathbf{w}} &= - \sum_n \left\{ y_n \frac{\sigma(\mathbf{w}^\top \mathbf{x}_n)[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]}{\sigma(\mathbf{w}^\top \mathbf{x}_n)} \mathbf{x}_n \right. \\ &\quad \left. - (1 - y_n) \frac{\sigma(\mathbf{w}^\top \mathbf{x}_n)[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]}{1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)} \mathbf{x}_n \right\} \\ &= - \sum_n \{y_n[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\mathbf{x}_n - (1 - y_n)\sigma(\mathbf{w}^\top \mathbf{x}_n)\mathbf{x}_n\} \\ &= \sum_n \underbrace{\{\sigma(\mathbf{w}^\top \mathbf{x}_n) - y_n\}}_{\text{Error of the } n\text{th training sample.}} \mathbf{x}_n \end{aligned}$$

input ↙

How to Handle More Complex Decision Boundaries?



- This data is not linearly separable...
- Use **non-linear basis functions** to add more features.

Solution to Overfitting: Regularization

- Add regularization term to be cross entropy loss function

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\} + \underbrace{\frac{1}{2} \lambda \|\mathbf{w}\|_2^2}_{\text{regularization}}$$

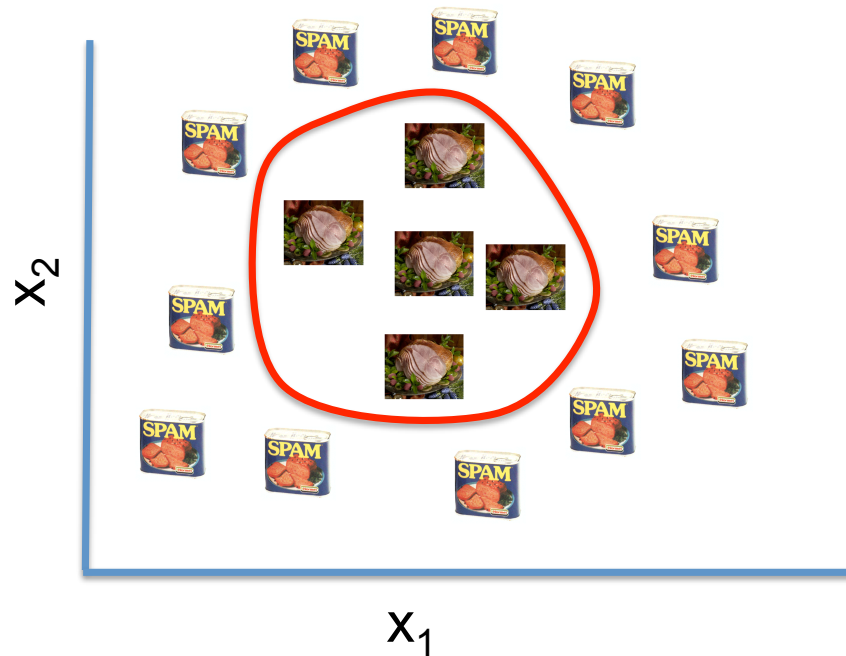
- Perform gradient descent on this regularized function
- Often, we do NOT regularize the bias term w_0

Solution to Overfitting: Regularization

- Add regularization term to be cross entropy loss function

$$\mathcal{E}(\mathbf{w}) = - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1-y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\} + \underbrace{\frac{1}{2} \lambda \|\mathbf{w}\|_2^2}_{\text{regularization}}$$

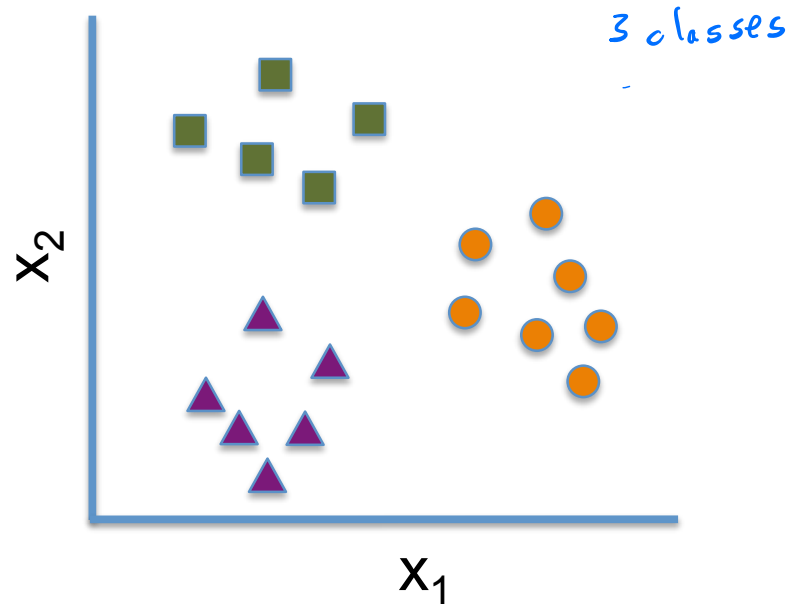
- Perform gradient descent on this regularized function
- Often, we do **NOT** regularize the bias term w_0



Multi-class Classification

What If There Are More than 2 Classes?

- Dog vs. cat. vs crocodile
- Movie genres (action, horror, comedy, ...)
- Part of speech tagging (verb, noun, adjective, ...)
- ...



Predict multiple classes/outcomes $C_1, C_2, \dots, C_M$: *M classes*

- Weather prediction: sunny, cloudy, raining, etc
- Optical character recognition: 10 digits + 26 characters (lower and upper cases) + special characters, etc.

M = number of classes

Predict multiple classes/outcomes $C_1, C_2, \dots, C_M$:

- Weather prediction: sunny, cloudy, raining, etc
- Optical character recognition: 10 digits + 26 characters (lower and upper cases) + special characters, etc.

M = number of classes

Methods we've studied for binary classification:

- Naïve Bayes
- Logistic regression

Do they generalize to multi-class classification?

Naïve Bayes Is Already Multi-class!

Formal Definition

Given a random vector $\mathbf{X} \in \mathbb{R}^K$ and a dependent variable $Y \in [C]$, the Naïve Bayes model defines the joint distribution

$$P(\mathbf{X} = \mathbf{x}, Y = c) = P(Y = c)P(\mathbf{X} = \mathbf{x} | Y = c) \quad (1)$$

$$= P(Y = c) \prod_{k=1}^K P(\text{word}_k | Y = c)^{x_k} \quad (2)$$

$$= \pi_c \prod_{k=1}^K \theta_{ck}^{x_k} \quad (3)$$

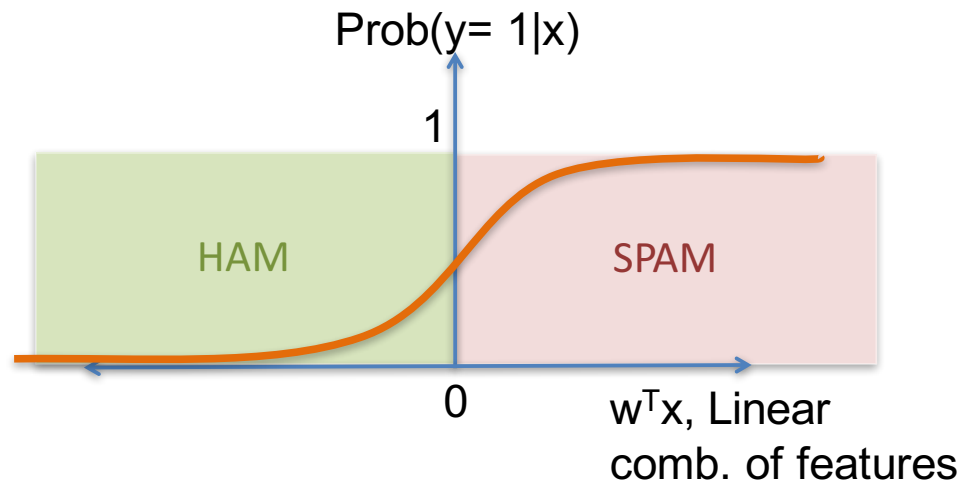
where x_k is the number of occurrences of the k th word, π_c is the prior probability of class c (which allows multiple classes!), and θ_{ck} is the weight of the k th word for the c th class.

Logistic Regression for Predicting Multiple Classes?

- The linear decision boundary that we optimized was specific to binary classification.
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) \geq 0.5$ declare $y = 1$ (spam)
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$ declare $y = 0$ (ham)

Logistic Regression for Predicting Multiple Classes?

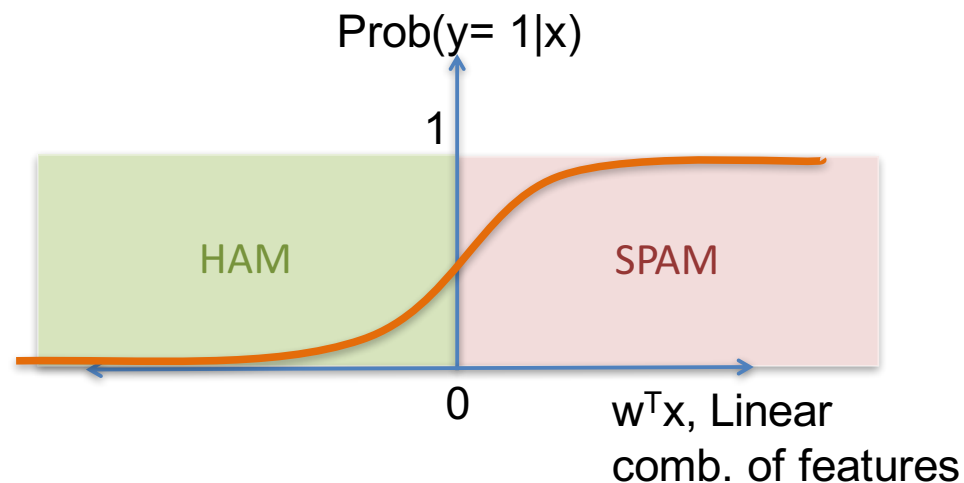
- The linear decision boundary that we optimized was specific to binary classification.
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) \geq 0.5$ declare $y = 1$ (spam)
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$ declare $y = 0$ (ham)
- How to extend it to multi-class classification?



$y = 1$ for spam, $y = 0$ for ham

Logistic Regression for Predicting Multiple Classes?

- The linear decision boundary that we optimized was specific to binary classification.
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) \geq 0.5$ declare $y = 1$ (spam)
 - If $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$ declare $y = 0$ (ham)
- How to extend it to multi-class classification?



$y = 1$ for spam, $y = 0$ for ham

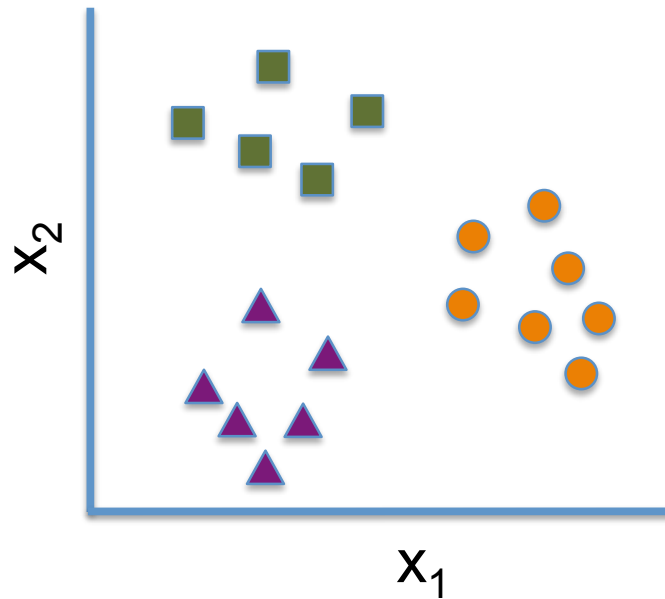
Idea: Express as multiple binary classification problems

The One-versus-Rest or One-versus-All Approach

- For each class c , change the problem into binary classification
 1. Relabel training data with label c , into POSITIVE (or '1').
 2. Relabel all the rest data into NEGATIVE (or '0').

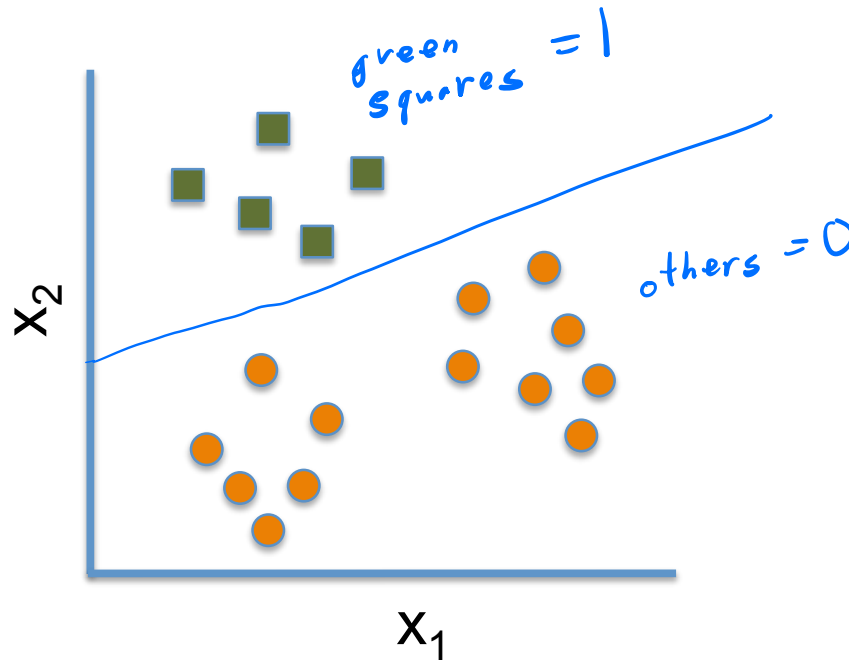
The One-versus-Rest or One-versus-All Approach

- For each class c , change the problem into binary classification
 1. Relabel training data with label c , into POSITIVE (or '1').
 2. Relabel all the rest data into NEGATIVE (or '0').
- Repeat this multiple times: Train C binary classifiers, using logistic regression to differentiate the two classes each time.



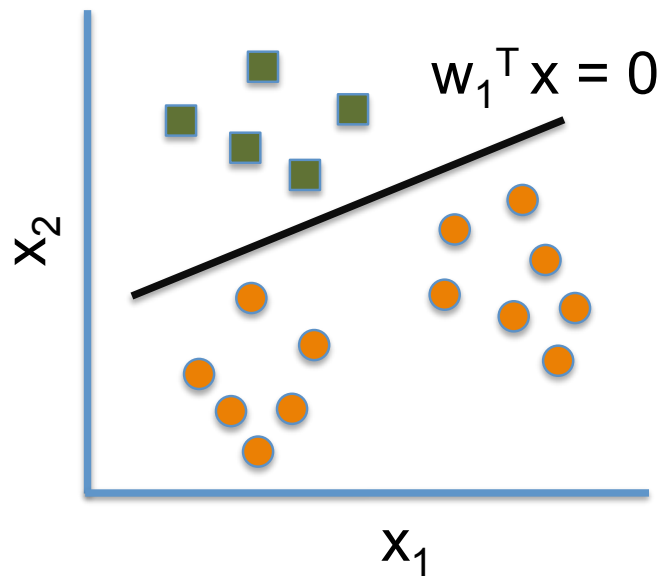
The One-versus-Rest or One-versus-All Approach

- For each class c , change the problem into binary classification
 1. Relabel training data with label c , into POSITIVE (or '1')
 2. Relabel all the rest data into NEGATIVE (or '0')
- Repeat this multiple times: Train C binary classifiers, using logistic regression to differentiate the two classes each time.



The One-versus-Rest or One-versus-All Approach

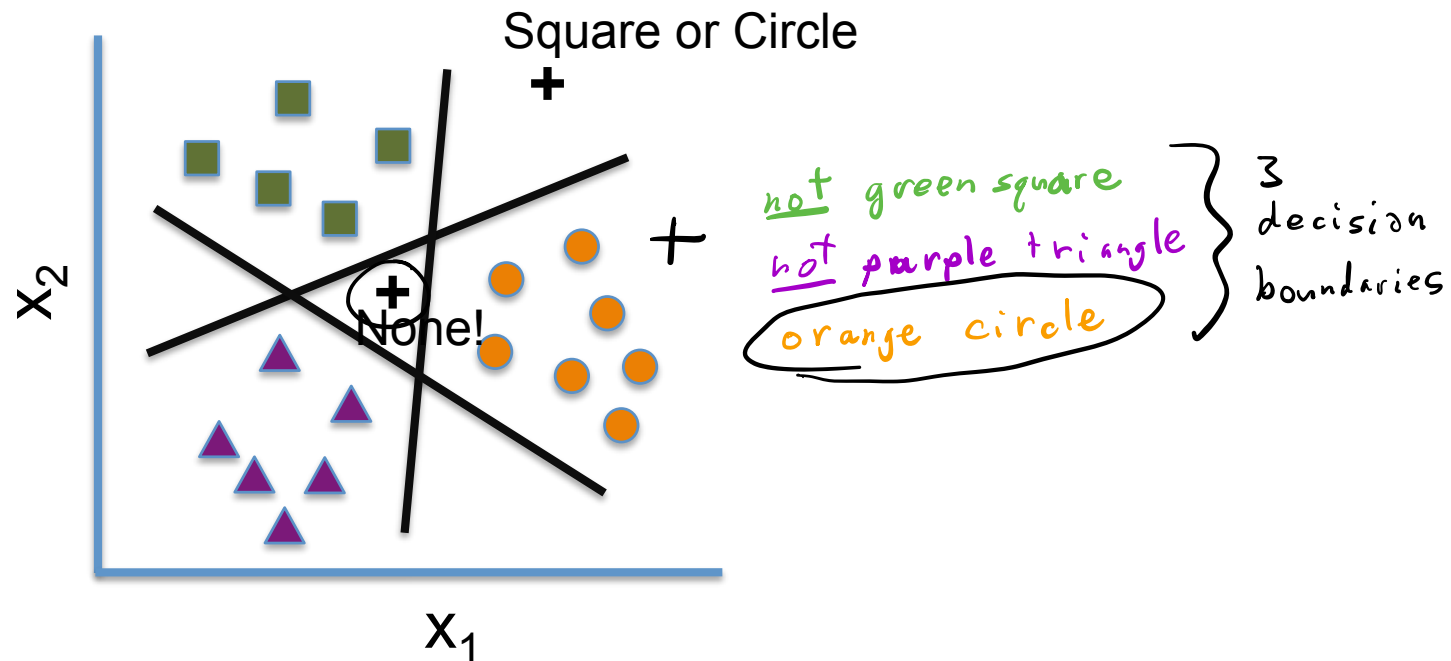
- For each class c , change the problem into binary classification
 1. Relabel training data with label c , into POSITIVE (or '1')
 2. Relabel all the rest data into NEGATIVE (or '0')
- Repeat this multiple times: Train C binary classifiers, using logistic regression to differentiate the two classes each time.



The One-versus-Rest or One-versus-All Approach

How to combine these linear decision boundaries?

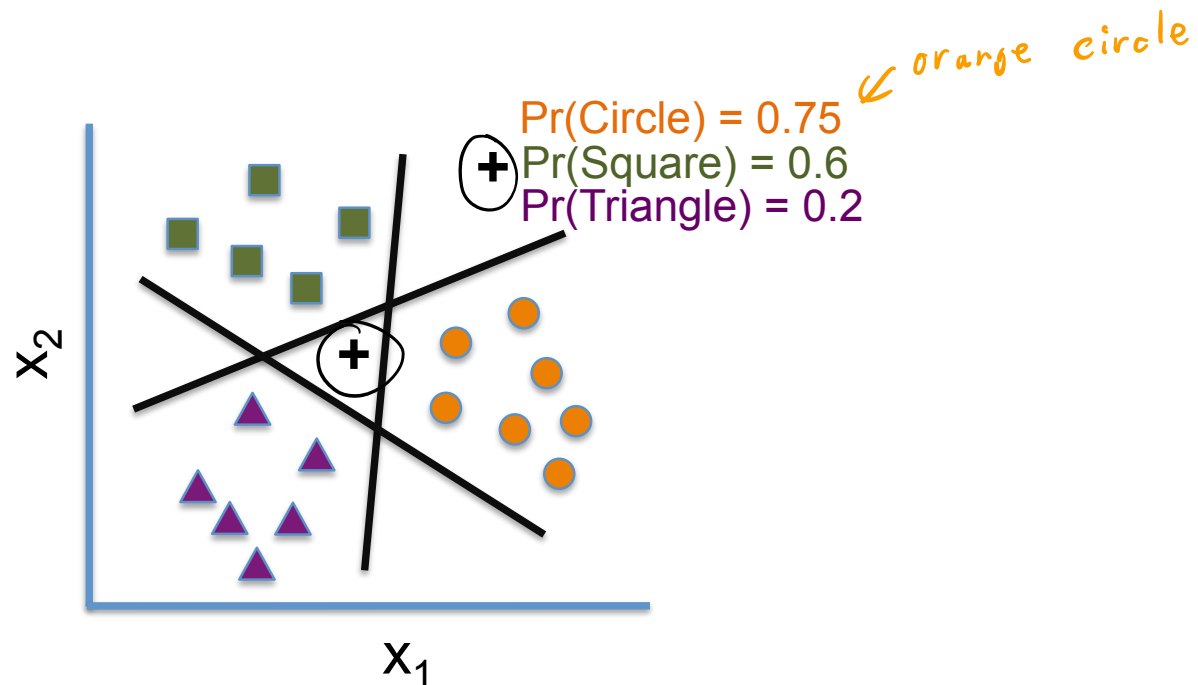
- There is ambiguity in some of the regions (the 4 triangular areas).
- How do we resolve this?



The One-versus-Rest or One-versus-All Approach

How to combine these linear decision boundaries?

- Use the confidence estimates $\Pr(y = 1|\mathbf{x}) = \sigma(\mathbf{w}_1^\top \mathbf{x})$,
... $\Pr(y = C|\mathbf{x}) = \sigma(\mathbf{w}_C^\top \mathbf{x})$

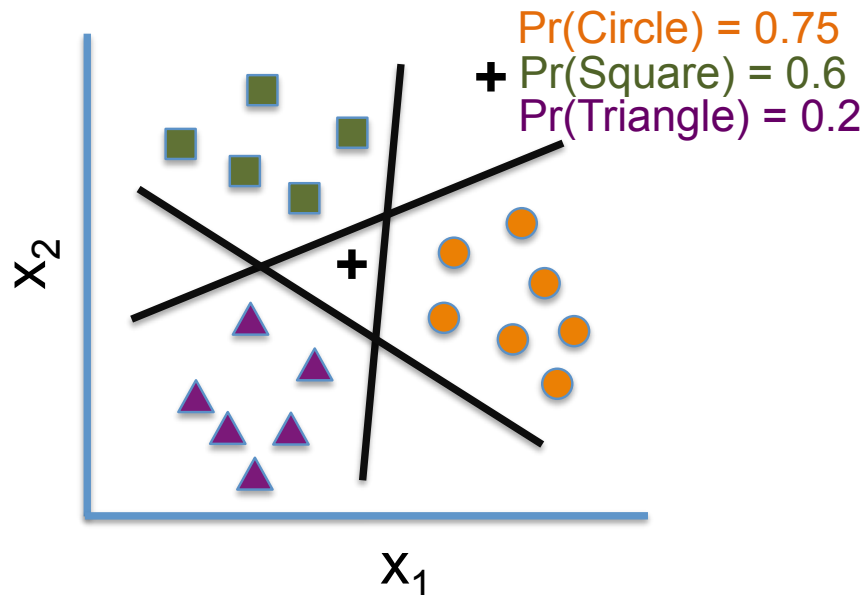


The One-versus-Rest or One-versus-All Approach

How to combine these linear decision boundaries?

- Use the **confidence estimates** $\Pr(y = 1|\mathbf{x}) = \sigma(\mathbf{w}_1^\top \mathbf{x})$,
... $\Pr(y = C|\mathbf{x}) = \sigma(\mathbf{w}_C^\top \mathbf{x})$
- Declare class c^* that maximizes

$$c^* = \arg \max_{\substack{c=1,\dots,C \\ C \text{ classifiers}}} \Pr(y = c|\mathbf{x}) = \sigma(\mathbf{w}_c^\top \mathbf{x})$$

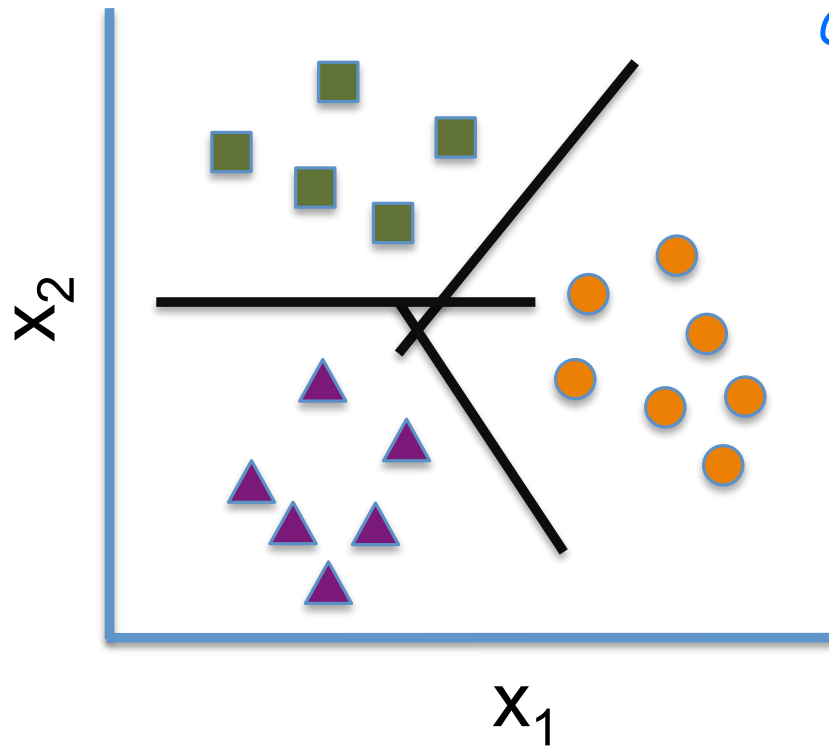


The One-versus-One Approach

- For each pair of classes c and c' , change the problem into binary classification.
 - Relabel training data with label c , into POSITIVE (or '1')
 - Relabel training data with label c' into NEGATIVE (or '0')
 - Disregard** all other data

pairs: $\frac{C(C-1)}{2}$

$C=3: 3 \text{ pairs}$



The One-versus-One Approach

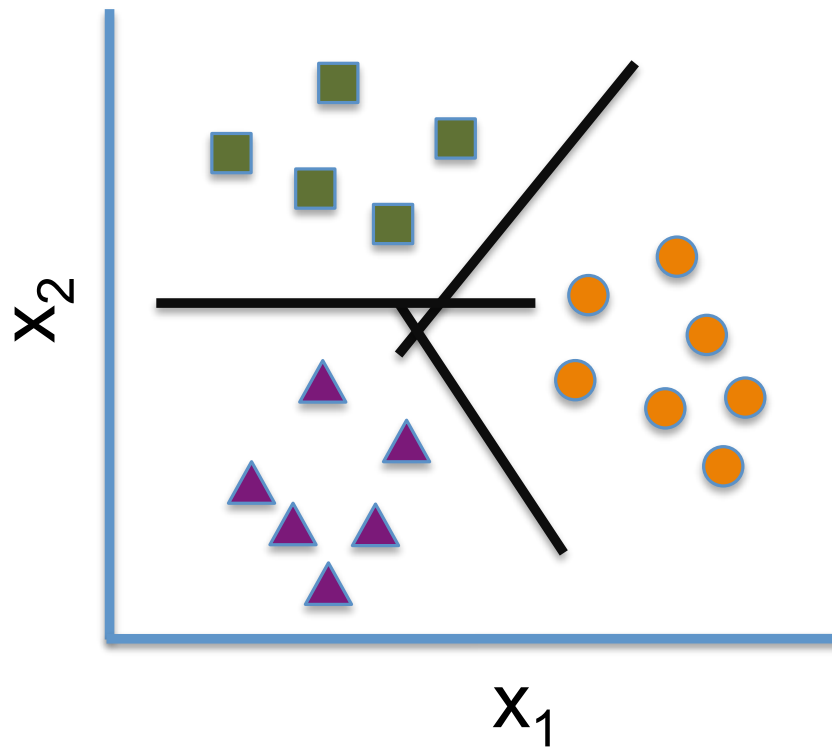
- How many binary classifiers for C classes?

The One-versus-One Approach

- How many binary classifiers for C classes?

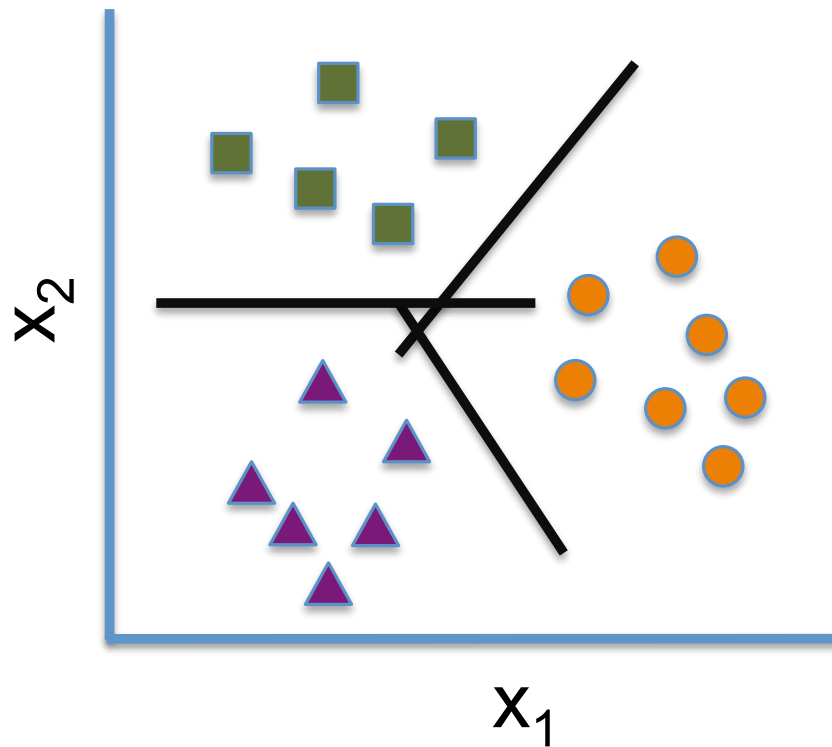
The One-versus-One Approach

- How many binary classifiers for C classes? $C(C - 1)/2$
- How to combine their outputs?



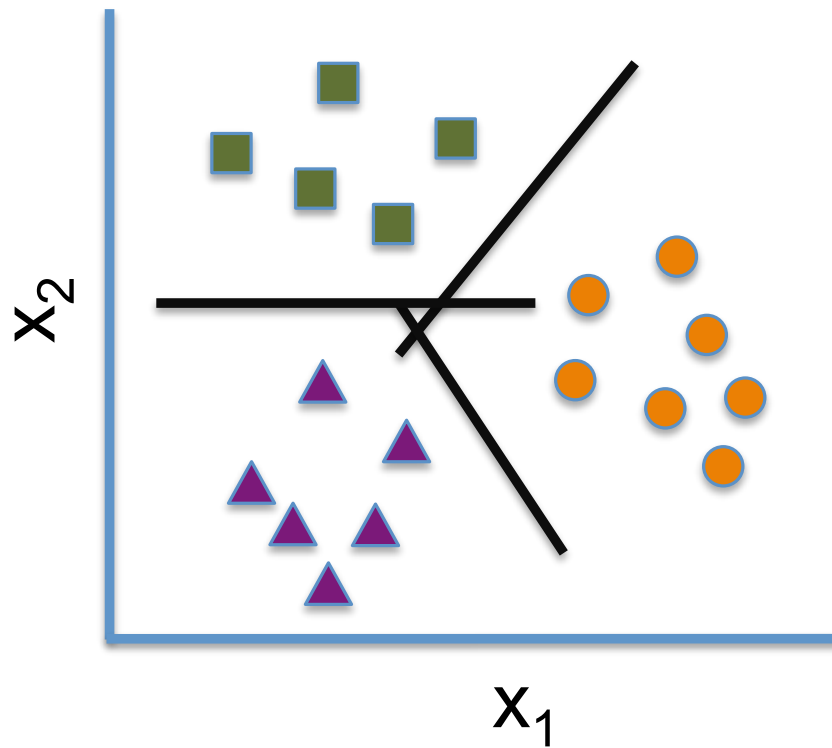
The One-versus-One Approach

- How many binary classifiers for C classes? $C(C - 1)/2$
- How to combine their outputs?
- Given $\mathbf{x}$, count the $C(C - 1)/2$ votes from outputs of all binary classifiers and declare the winner as the predicted class.



The One-versus-One Approach

- How many binary classifiers for C classes? $C(C - 1)/2$
- How to combine their outputs?
- Given $\mathbf{x}$, count the $C(C - 1)/2$ votes from outputs of all binary classifiers and declare the winner as the predicted class.
- Use confidence scores to resolve ties.



Contrast These Approaches

Number of binary classifiers to be trained

- One-versus-All: C classifiers.
- One-versus-One: $C(C - 1)/2$ classifiers – bad if C is large

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

- **Hierarchical classification** – we will see this in decision trees

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

- **Hierarchical classification** – we will see this in decision trees
- **Multinomial logistic regression** – directly output probabilities of y being in each of the C classes.

Multinomial Logistic Regression

Intuition:

from the decision rule of our Naïve Bayes classifier

$$\begin{aligned} y^* &= \arg \max_c P(y = c | \mathbf{x}) = \arg \max_c \log p(\mathbf{x} | y = c) p(y = c) \\ &= \arg \max_c \log \pi_c + \sum_k x_k \log \theta_{ck} = \arg \max_c \mathbf{w}_c^\top \mathbf{x} \end{aligned}$$

Multinomial Logistic Regression

Intuition:

from the decision rule of our Naïve Bayes classifier

$$\begin{aligned} y^* &= \arg \max_c P(y = c | \mathbf{x}) = \arg \max_c \log p(\mathbf{x} | y = c) p(y = c) \\ &= \arg \max_c \log \pi_c + \sum_k x_k \log \theta_{ck} = \arg \max_c \mathbf{w}_c^\top \mathbf{x} \end{aligned}$$

Essentially, we are comparing

$$\mathbf{w}_1^\top \mathbf{x}, \mathbf{w}_2^\top \mathbf{x}, \dots, \mathbf{w}_C^\top \mathbf{x}$$

with **one** for each category.

So, can we define the following conditional model?

$$P(y = c|\mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$

So, can we define the following conditional model?

$$P(y = c|\mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$

This would **not** work because:

$$\sum_c P(y = c|\mathbf{x}) = \sum_c \sigma[\mathbf{w}_c^\top \mathbf{x}] \neq 1,$$

so each summand can be any number (independently) between 0 and 1.

So, can we define the following conditional model?

$$P(y = c|\mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$

This would **not** work because:

$$\sum_c P(y = c|\mathbf{x}) = \sum_c \sigma[\mathbf{w}_c^\top \mathbf{x}] \neq 1,$$

so each summand can be any number (independently) between 0 and 1.

But we are close!

Learn the C linear models jointly to ensure this property holds!

Multinomial Logistic Regression

- **Model:** For each class c , we have a parameter vector $\mathbf{w}_c$ and model the posterior probability as:

$$P(c|\mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x}}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}}} \quad \leftarrow \quad \textit{This is called the softmax function.}$$

Multinomial Logistic Regression

- **Model:** For each class c , we have a parameter vector $\mathbf{w}_c$ and model the posterior probability as:

$$P(c|\mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x}}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}}} \quad \leftarrow \quad \textit{This is called the softmax function.}$$

- **Decision boundary:** Assign $\mathbf{x}$ with the label that is the maximum of posterior:

$$\arg \max_c P(c|\mathbf{x}) \rightarrow \arg \max_c \mathbf{w}_c^\top \mathbf{x}.$$

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the **winning** class label 1.

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the **winning** class label 1.

Softmax translates these scores into well-formed conditional probabilities

$$P(y = 1|\mathbf{x}) = \frac{e^{100}}{e^{100} + e^{50} + e^{-20}} < 1$$

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the **winning** class label 1.

Softmax translates these scores into well-formed conditional probabilities

$$P(y = 1|\mathbf{x}) = \frac{e^{100}}{e^{100} + e^{50} + e^{-20}} < 1$$

- Preserves relative ordering of scores.
- Maps scores to values between 0 and 1 that also sum to 1.

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

We will change y_n to $\mathbf{y}_n = [y_{n1} \ y_{n2} \ \cdots \ y_{nC}]^\top$, a C -dimensional vector using 1-of- C encoding.

$$y_{nc} = \begin{cases} 1 & \text{if } y_n = c \\ 0 & \text{otherwise} \end{cases}$$

Ex: if $y_n = 2$, then, $\mathbf{y}_n = [0 \ \mathbf{1} \ 0 \ 0 \ \cdots \ 0]^\top$.

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

We will change y_n to $\mathbf{y}_n = [y_{n1} \ y_{n2} \ \cdots \ y_{nC}]^\top$, a C -dimensional vector using 1-of- C encoding.

$$y_{nc} = \begin{cases} 1 & \text{if } y_n = c \\ 0 & \text{otherwise} \end{cases}$$

Ex: if $y_n = 2$, then, $\mathbf{y}_n = [0 \ \mathbf{1} \ 0 \ 0 \ \cdots \ 0]^\top$.

$$\Rightarrow \sum_n \log P(y_n | \mathbf{x}_n) = \sum_n \log \prod_{c=1}^C P(c | \mathbf{x}_n)^{y_{nc}} = \sum_n \sum_c y_{nc} \log P(c | \mathbf{x}_n)$$

Cross-entropy Error Function

Definition: negative log-likelihood

$$\begin{aligned}\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) &= - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n) \\ &= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)\end{aligned}$$

Cross-entropy Error Function

Definition: negative log-likelihood

$$\begin{aligned}\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) &= - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n) \\ &= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)\end{aligned}$$

Properties of cross-entropy

- Convex in the $\mathbf{w}$ vectors, therefore local minimum = global optimum

Cross-entropy Error Function

Definition: negative log-likelihood

$$\begin{aligned}\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) &= - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n) \\ &= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)\end{aligned}$$

Properties of cross-entropy

- Convex in the $\mathbf{w}$ vectors, therefore local minimum = global optimum
- Optimization requires numerical procedures, analogous to those used for binary logistic regression.

You Should Know

- How to generalize logistic regression to handle nonlinear decision boundaries.
- How to handle multiclass classification: one-versus-all, one-versus-one, multinomial regression.
- How to measure (binary) classification accuracy